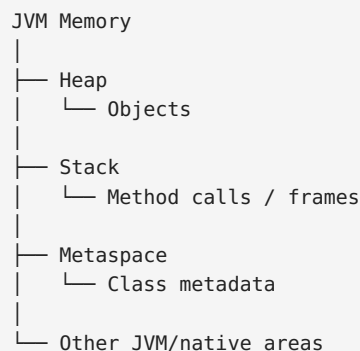


Memory Management: Heap vs Stack, Garbage Collection

1. How Java Memory Works

When a Java application runs, the JVM manages several memory areas.

For your interview, focus mainly on:



The two most important concepts are **Heap** and **Stack**.

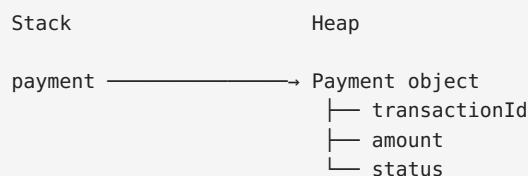
2. Heap

The **heap** is the main memory area where Java objects and arrays are allocated.

Example:

```
Payment payment = new Payment();
```

Conceptually:



The variable `payment` is a reference used by the executing method, while the `Payment` object itself is allocated on the heap.

Heap characteristics

- Shared among threads
 - Stores objects/arrays
 - Managed by Garbage Collector
 - Usually much larger than an individual thread's stack
-

3. Stack

Each thread has its own **JVM stack**.

Whenever a method is called, a **stack frame** is created.

Example:

```
public void processPayment() {  
    int amount = 1000;  
    Payment payment = new Payment();  
}
```

Conceptually:

Thread Stack

```
processPayment()  
-----  
amount = 1000  
payment -----> Heap object  
-----
```

When the method returns, its stack frame is removed.

Stack characteristics

- Each thread has its own stack
- Stores stack frames for method invocations
- Contains local variables/references and execution information
- Automatically cleaned when methods return
- Stack access is very fast

4. Heap vs Stack

This is a guaranteed interview topic.

Heap	Stack
Shared across threads	Each thread has its own
Stores objects/arrays	Stores method frames/local execution state
Managed largely by GC	Frames removed as methods return
Larger	Smaller per thread
Can suffer GC pressure	Can suffer stack overflow
Object lifetime can extend beyond method	Frame normally exists during method execution

Easy memory trick

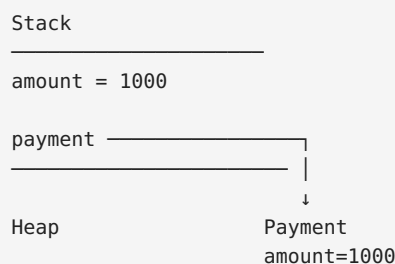
Stack = method execution Heap = objects

5. Example

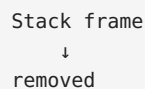
Consider:

```
public Payment createPayment() {  
    int amount = 1000;  
    Payment payment = new Payment();  
    payment.setAmount(amount);  
    return payment;  
}
```

Conceptually:



When `createPayment()` returns:



But the `Payment` object can remain on the heap because the returned reference may still be reachable by the caller.

6. Important Correction: References Aren't “Always on Stack”

You may hear:

“Objects are in heap and references are in stack.”

That’s a useful simplified interview explanation, but it is not a complete description of JVM implementation.

The JVM uses optimizations such as **escape analysis**, and the JIT compiler may optimize allocations and scalar-replace objects in some situations.

For your interview, say:

“Conceptually, local references are associated with stack frames while objects are allocated on the heap, although JVM/JIT optimizations can change the physical implementation.”

That is a technically stronger answer.

7. What is Garbage Collection?

Java uses **automatic memory management**.

When objects are no longer reachable from the application's live references, they can eventually be reclaimed by the Garbage Collector.

Example:

```
Payment payment = new Payment();  
  
payment = null;
```

The original object may become eligible for garbage collection if no other reference points to it.

```
payment —> Payment object  
  
payment = null  
  
Payment object  
  ↓  
No reachable reference  
  ↓  
Eligible for GC
```

Important:

Eligible for GC does not mean immediately removed.

8. How GC Determines What Can Be Collected

The GC primarily works from **GC roots** and traces reachable objects.

Common GC roots include:

- live thread stacks
- static references
- JNI/native references
- other JVM-managed roots

Conceptually:

```
GC Root  
  ↓  
Object A  
  ↓  
Object B  
  
Object C ← unreachable  
Object D ← unreachable
```

A and B are reachable.

C and D can eventually be reclaimed.

9. Reachable vs Unreachable

This is better than thinking simply:

“Set reference to null = garbage collected.”

Example:

```
Payment p1 = new Payment();
Payment p2 = p1;

p1 = null;
```

Is the Payment object eligible for GC?

No.

Because:

```
p2 —————> Payment
```

It is still reachable.

10. Can an Object Become Garbage Without Setting It to Null?

Absolutely.

Example:

```
public void test() {
    Payment payment = new Payment();

} // method ends
```

After the method returns, if nothing else references that object, it can become unreachable.

You don't need:

```
payment = null;
```

Usually that would be unnecessary.

11. Garbage Collection Process

A simplified conceptual model:

```
Objects allocated
  ↓
Some become unreachable
```

```
↓
GC identifies unreachable objects
↓
Memory reclaimed
```

Modern collectors use much more sophisticated algorithms.

Historically and conceptually, interview explanations often use:

```
Young Generation
Old Generation
```

and concepts such as:

```
Minor GC
Major/Full GC
```

12. Young vs Old Generation

A common generational model:

```
Heap
|
├── Young Generation
│   ├── Eden
│   └── Survivor spaces
└── Old Generation
```

New objects are typically allocated into the young generation.

Objects that survive multiple collections may be promoted to the old generation.

Conceptually:

```
new objects
↓
Eden
↓
Survivor
↓
Old Generation
```

Not every modern GC implementation should be described as if it follows this exact model in all details, but this is still useful for understanding generational GC.

13. Why Generational GC?

The basic observation is:

■ **Most objects die young.**

For example:

```
PaymentResponse response = ...
```

Many request-scoped temporary objects may live only for a short time.

Therefore collecting young objects frequently can be efficient.

14. Minor GC

A **young-generation collection** is often called a Minor GC in generational collectors.

Conceptually:

```
Eden
↓
GC
↓
Dead objects removed
Live objects moved/promoted
```

It is generally much cheaper than collecting a very large old generation, although exact behavior depends on the collector.

15. Old Generation

Objects that survive long enough may be promoted to older regions.

Examples could include:

- long-lived caches
- application metadata
- persistent in-memory data structures

If old-generation occupancy becomes problematic, GC may require more expensive work.

16. Stop-the-World

Some GC phases may require application threads to pause.

This is called a **Stop-The-World (STW) pause**.

Conceptually:

```
Application threads
↓
PAUSE
↓
GC work
↓
RESUME
```

Modern collectors try to minimize pause times and perform much work concurrently, but **some stop-the-world pauses still exist**.

This is important for latency-sensitive applications such as payment systems.

17. Common Java Garbage Collectors

For modern Java, you should know at least:

- **G1 GC**
- **ZGC**
- **Shenandoah**

G1 is a common general-purpose collector in modern JDKs.

ZGC and Shenandoah are designed for very low pause times, especially useful for large heaps/latency-sensitive workloads.

Don't memorize exact behavior as universal across every Java version; GC implementations evolve.

18. What Causes `OutOfMemoryError`?

A common misconception:

“`OutOfMemoryError` means GC doesn't work.”

Not necessarily.

It can happen when the JVM cannot allocate needed memory despite garbage collection.

Example:

```
List<byte[]> data = new ArrayList<>();

while (true) {
    data.add(new byte[1024 * 1024]);
}
```

Objects remain strongly reachable through `data`.

Therefore GC cannot reclaim them.

Eventually:

```
java.lang.OutOfMemoryError: Java heap space
```

19. Memory Leak in Java

Java has garbage collection, but **memory leaks can still happen**.

A Java memory leak means:

Objects are no longer logically needed but remain reachable, so GC cannot reclaim them.

Example:

```
static List<Payment> cache = new ArrayList<>();

public void add(Payment payment) {
    cache.add(payment);
}
```

If this list grows forever:

```
GC Root
  ↓
static cache
  ↓
Payment objects
Payment objects
Payment objects
...
```

The objects remain reachable.

Therefore:

```
GC runs
  ↓
Objects still reachable
  ↓
Cannot reclaim
  ↓
Heap keeps growing
```

20. Common Causes of Java Memory Leaks

Know these:

1. Static collections

```
static List<Object> list;
```

2. Unbounded caches

```
cache grows forever
```

3. Listeners/callback registrations not removed

4. ThreadLocal misuse

Especially with long-lived thread pools.

5. Objects unintentionally kept by long-lived references

For example, global registries.

21. StackOverflowError

Stack and heap have different failure modes.

Infinite recursion:

```
public void test() {  
    test();  
}
```

results in:

```
StackOverflowError
```

Conceptually:

```
test()  
↓  
test()  
↓  
test()  
↓  
...  
↓  
Stack exhausted
```

22. StackOverflowError vs OutOfMemoryError

	StackOverflowError	OutOfMemoryError
Typical cause	Excessive recursion/deep call stack	JVM can't allocate required memory
Area	Thread stack	Often heap, but OOME can have other causes
Example	Infinite recursion	Unbounded object allocation

23. System.gc()

You may be asked:

“Can we force garbage collection?”

You can request it:

```
System.gc();
```

But:

It is only a request/hint to the JVM, not a guarantee that GC will run immediately.

You should not build application correctness around it.

24. Does GC Run When an Object Goes Out

of Scope?

Not necessarily.

Example:

```
public void process() {  
    Payment p = new Payment();  
}
```

When the method finishes:

```
p reference gone  
  ↓  
object may become unreachable  
  ↓  
eligible for GC
```

But GC may happen later, not immediately.

25. `finalize()` — Important Modern Interview Point

Older Java versions had `finalize()` as a mechanism associated with object cleanup.

You should know:

❗ **Do not use `finalize()` for resource management.**

It has been deprecated for removal.

Use:

- try-with-resources
- `AutoCloseable`
- explicit resource management

Example:

```
try (FileInputStream input =  
    new FileInputStream("data.txt")) {  
    ...  
}
```

26. Heap Memory in a Spring Boot Application

Think about your BPS application.

Suppose 1000 requests arrive:

```
BPS
↓
1000 requests
↓
1000 sets of request objects
↓
DTOs
Entities
Responses
Temporary objects
```

Many are short-lived.

```
Request
↓
Create DTO
↓
Process payment
↓
Create response
↓
Request completes
↓
Objects become unreachable
↓
GC eventually reclaims them
```

If you're creating huge objects unnecessarily or retaining references in static caches, heap usage can grow.

27. How Would You Diagnose High Heap Usage?

This is an excellent **3-year production interview question**.

Don't just say:

▮ "Increase `-Xmx`."

A better approach:

```
High memory usage
↓
Check JVM metrics
↓
Check GC behavior
↓
Check heap usage after GC
↓
Look for continuously growing old-gen
↓
Take heap dump
↓
Analyze retained objects
↓
Identify leak / excessive allocation
↓
Fix root cause
```

Tools you should know by name:

- **JVisualVM**
- **JConsole**
- **Java Flight Recorder (JFR)**
- **jcmd**
- heap dump analyzers such as Eclipse MAT

28. Important Distinction: High Allocation vs Memory Leak

Suppose:

```
Heap usage:
20% → 70% → 90%
GC
90% → 25%
```

That could simply indicate **high temporary allocation**.

But:

```
20% → 70% → 90%
GC
90% → 80%
GC
80% → 85%
GC
...
```

If the post-GC baseline continually increases, that can indicate **retained objects / memory leak or insufficient heap for the workload**.

This is a useful production-debugging concept.

29. JVM Options

You may encounter:

```
-Xms512m
-Xmx2g
```

-Xms

Initial/minimum heap size setting.

-Xmx

Maximum heap size.

Example:

```
-Xms512m  
-Xmx2g
```

means the JVM starts with a heap target of 512 MB and can grow up to a 2 GB maximum under that configuration.

Exact JVM memory behavior is more nuanced than treating these as fixed physical allocations.

30. Stack Size

You may also see:

```
-Xss1m
```

This controls the Java thread stack size.

Too small:

```
deep call stack  
→ StackOverflowError
```

Too large combined with many threads:

```
many threads × large stacks  
→ significant native memory consumption
```

This connects directly to your previous **multithreading** topic.

31. Garbage Collection and Multithreading

Remember:

```
Heap  
↓  
Shared among threads
```

while:

```
Thread 1 → own stack  
Thread 2 → own stack  
Thread 3 → own stack
```

Therefore:

```
Many threads  
+  
Large -Xss  
+  
Large heap
```

can create substantial overall memory consumption.

32. Common Interview Questions

Heap/Stack

1. What is heap memory?
2. What is stack memory?
3. Heap vs stack?
4. Why is stack thread-specific?
5. Are objects stored in stack or heap?
6. Where are local variables stored?
7. What is a stack frame?
8. What causes `StackOverflowError`?
9. What causes `OutOfMemoryError`?

Garbage Collection

10. What is garbage collection?
 11. How does GC know an object is garbage?
 12. What are GC roots?
 13. Can we force GC?
 14. What is `System.gc()`?
 15. What is generational GC?
 16. Young vs old generation?
 17. What is Minor GC?
 18. What is Stop-The-World?
 19. What is a memory leak in Java?
 20. Can Java have memory leaks even with GC?
 21. What are common causes of memory leaks?
 22. What is G1 GC?
 23. What is ZGC?
 24. How would you investigate high memory usage?
-

33. Very Likely Cross-Questions

Q: Is garbage collection immediate when an object becomes unreachable?

| No. The object becomes eligible for collection, but the JVM decides when GC runs.

Q: Can you force garbage collection?

| `System.gc()` can request GC, but it doesn't guarantee immediate collection and shouldn't be relied on for application correctness.

Q: Can Java have memory leaks?

| Yes. If objects are logically unused but still strongly reachable—for example through a static collection, unbounded cache or improperly managed `ThreadLocal`—GC cannot reclaim them.

Q: What's the difference between heap and stack?

Heap is shared memory used primarily for objects and arrays and is managed by the GC. Each thread has its own stack containing method frames and execution state.

Q: What happens when a method returns?

Its stack frame is removed. Objects created during the method don't necessarily disappear; if they are still reachable from elsewhere, they remain alive.

34. Project-Based Question for BPS

An interviewer may ask:

"Suppose BPS suddenly starts throwing `OutOfMemoryError` in production. What would you do?"

A good answer:

"First I'd determine whether the issue is heap exhaustion, excessive allocation, or another memory category. I'd check JVM heap/GC metrics and look at memory after GC. If the post-GC baseline keeps increasing, I'd investigate retained objects and possible memory leaks. I'd capture and analyze a heap dump to identify which objects are consuming memory and what references are retaining them. I'd also check recent code/configuration changes, caches, collections and ThreadLocal usage. Increasing `-Xmx` might provide temporary headroom, but I'd treat it as mitigation rather than the root-cause fix."

That's a **3-year-level production answer**.

35. One Important Modern Java Point

Don't describe JVM memory as only:

Heap + Stack

At your experience level, know that there are additional areas such as:

Heap
Thread Stacks
Metaspace
Code Cache
Native/off-heap memory

For example, **Metaspace** stores class metadata and replaced the old PermGen model in Java 8.

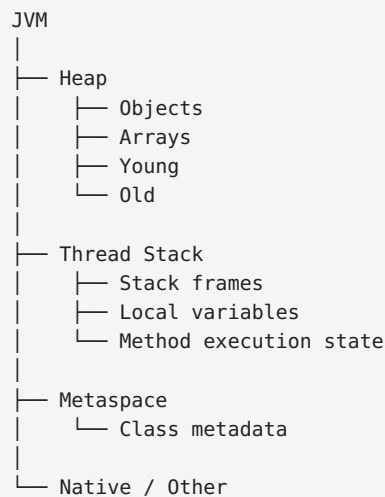
So if an interviewer asks:

"Is all JVM memory inside the Java heap?"

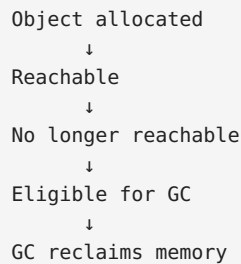
Answer:

No. Heap is only one part of JVM/process memory. Thread stacks, metaspace, code cache and native/off-heap allocations also consume memory.

36. Interview Memory Map



Garbage Collection:



37. What You Should Know for a 3-Year Interview

Must know

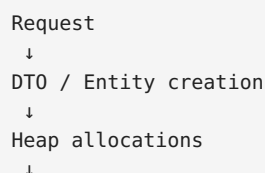
Heap vs Stack Stack frame Object reachability GC roots Garbage collection basics
Young/Old generation Minor GC Stop-the-world OutOfMemoryError StackOverflowError
Java memory leaks `System.gc()` G1 basics

Good to know

Metaspace JFR/JVisualVM/heap dumps ZGC/Shenandoah basics `-Xms` , `-Xmx` , `-Xss` Escape
analysis at a conceptual level

Especially connect it to your project

For BPS, understand:



```
Processing
↓
Request completes
↓
Objects become unreachable
↓
GC
```

And be able to explain:

“If memory usage keeps increasing, how would you distinguish normal allocation pressure from a memory leak?”

That question ties together **Java memory management + production debugging + your 3-year developer experience**, so it is worth preparing very well.